IS LAB B	Martynenko Mykhailo Album 55625 Grupa 436	**Wersja 2026-09-14**

JS I DOM NA PRZYKŁADZIE LISTY TODO

SPIS TREŚCI

Spis treści....1
Cel zajęć....1
Rozpoczęcie....1
Uwaga....2
Wymagania....2
Strona HTML....3
Klasa Todo....3
Dodawanie pozycji listy....4
Usuwanie pozycji listy....4
Edycja pozycji listy....5
Odczyt / Zapis LocalStorage....5
Wyszukiwanie....6
Commit projektu do GIT....7
Podsumowanie....7

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- przemieszczania się po drzewie DOM;
- dodawania, usuwania, edytowania elementów drzewa DOM;
- rozdzielania logiki biznesowej od warstwy wizualizacji.

W praktycznym wymiarze utworzona zostanie dynamiczna lista czynności do zrobienia (lista To Do).

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie metod przemieszczania się po drzewie DOM.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w poprzednio, wejdź do właściwości dokumentu i zaktualizuj niestandardowe pola.

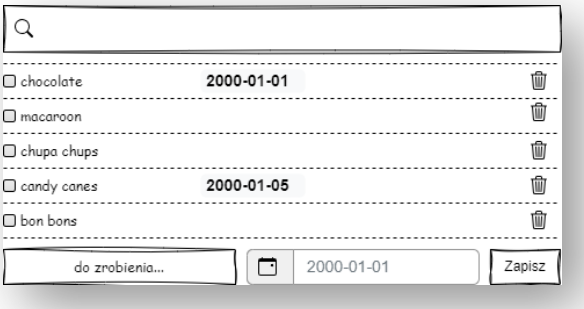
WYMAGANIA

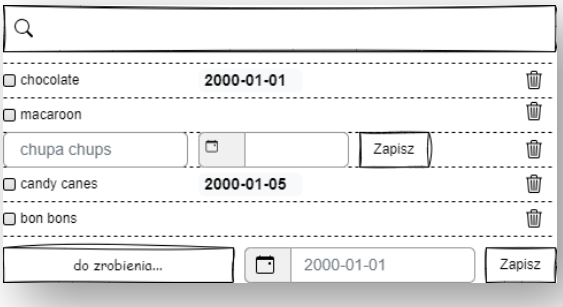
Szablon plików wymaganych w LAB B został dostarczony razem z instrukcją.

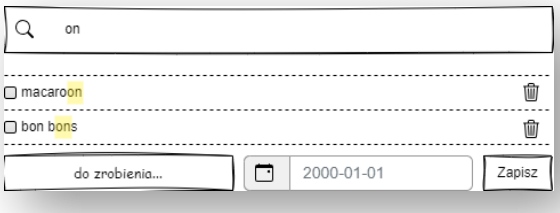
W ramach LAB B przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS i stylami z zewnętrznego pliku CSS;
- lista zadań;
- na dole listy pole tekstowe do dodawania nowych zadań, pole typu data/czas do określenia terminu wykonania zadania, przycisk dodawania zadania;
- walidacja nowych zadań: co najmniej 3 znaki, nie więcej niż 255 znaków, data musi być pusta albo w przyszłości;
- na górze listy pole wyszukiwarki;
- po wpisaniu w wyszukiwarkę co najmniej 2 znaków na liście wyświetlają się wyłącznie pozycje zawierające wpisaną w wyszukiwarkę frazę;
- wyszukiwana fraza zostaje wyróżniona w każdym wyniku wyszukiwania;
- kliknięcie na dowolną pozycję listy zmienia ją w pole edycji; kliknięcie poza pozycję listy zapisuje zmiany
- obok każdej pozycji listy znajduje się przycisku Usuń / Śmietnik
- wpisy na liście zapisują się do Local Storage
- po odświeżeniu strony lista wypełnia się wpisami z Local Storage

Mockupy (traktować jako inspirację a nie wymaganie!):







Strona HTML

Prace rozpocznij od rozpakowania dostarczonego szablonu projektu jako katalog `I:\IS\lab-b`. Ustaw w tytule swoje imię i nazwisko oraz numer indeksu po czym zacommituj zmiany, żeby mieć dobry punkt startowy, jakby trzeba było cofać zmiany.

Kodowanie zacznij od implementacji HTML z danymi wpisanymi „na sztywno”. Upewnij się, że wstawione zostały wszystkie wymagane elementy – pole wyszukiwarki, lista, pole dodawania, przycisk usuwania. To laboratorium koncentruje się na JS, więc może być ładne, ale nie musi. **Nie trać za dużo czasu na CSS** – to jest laboratorium z JS.

Wstaw zrzut ekranu przedstawiający stronę HTML z polem wyszukiwarki, listą, polem dodawania, przyciskami usuwania:



Punkty:	0	1

Klasa Todo

Pierwszym instynktem może być chęć dodania zachowań bezpośrednio do elementów listy w drzewie DOM. Chociaż na krótką metę wydaje się być to najprostsze rozwiązanie, za chwilę okaże się krótkowzroczne i trudne do implementacji przy kolejnych punktach 😊

Najlepszym sposobem rozwiązania tego laboratorium jest utworzenie klasy Todo (albo po prostu obiektu z kilkoma metodami). Bez względu na przyjętą strategię, należy w tym nowoutworzonym bycie utworzyć tablicę `tasks` oraz metodę `draw()`, która wyczyści `div` z obecną wizualizacją zadań do zrobienia i wygeneruje ją na nowo na podstawie tablicy `tasks`.

W celu sprawdzenia poprawności działania, najlepiej dostać się do tablicy `tasks` i edytować jej zawartość, po czym ręcznie wywołać metodę `draw()`. Jeśli zawartość listy wyrenderuje się na nowo poprawnie – możemy iść dalej!

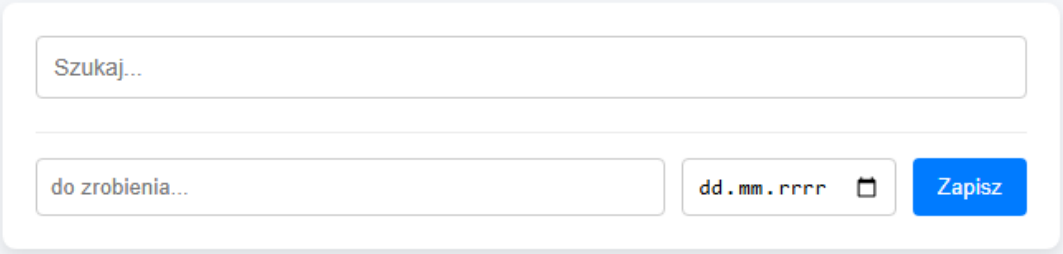
Zaimplementuj dodawanie, usuwanie, edycję pozycji listy – wszystko modyfikujące tablicę `tasks` i wywołujące na koniec metodę `draw()`.

Jak to testować? Na końcu swojego kodu JS stwórz instancję swojej klasy. Tak utworzony obiekt, przypisz do `document.todo`. Teraz w narzędziach deweloperskich przeglądarki, w zakładce konsoli możesz wykonać każdą metodę manualnie! Wystarczy wpisać: `document.todo.draw()` albo `document.todo.add(……)` itp.

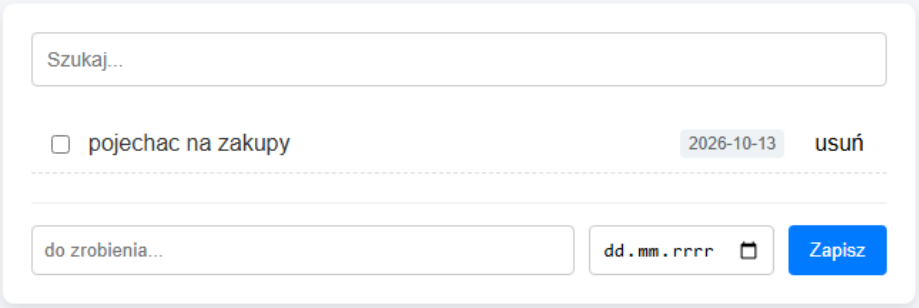
Gdy wszystkie metody operujące na modelu Todo będą gotowe, możesz podpiąć zachowania do drzewa DOM za pomocą `addEventListener` albo atrybutów `onclick`, `onsubmit`, `onfocus`, `onblur` itp.

DODAWANIE POZYCJI LISTY

Wstaw zrzut ekranu listy przed dodaniem nowego zadania:



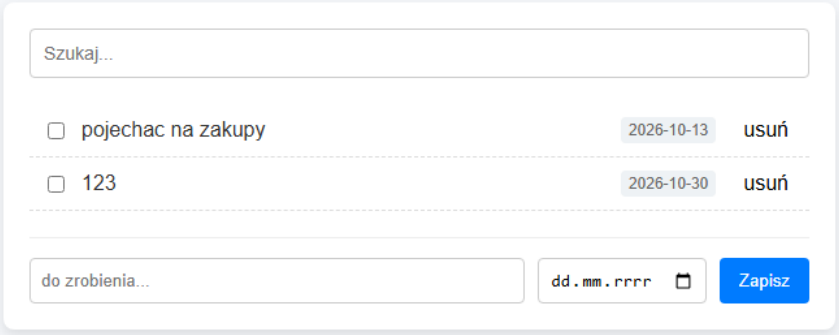
Wstaw zrzut ekranu listy po dodaniu nowego zadania:



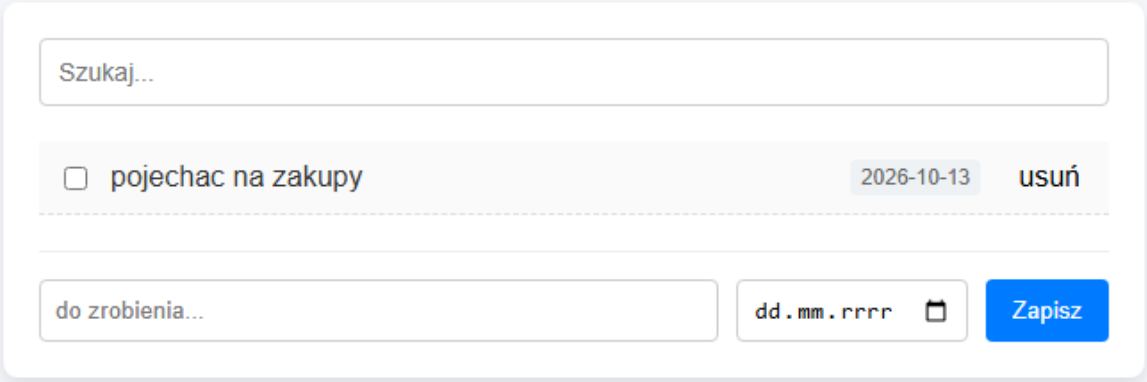
Punkty:	0	1

USUWANIE POZYCJI LISTY

Wstaw zrzut ekranu listy przed usunięciem wybranego zadania:



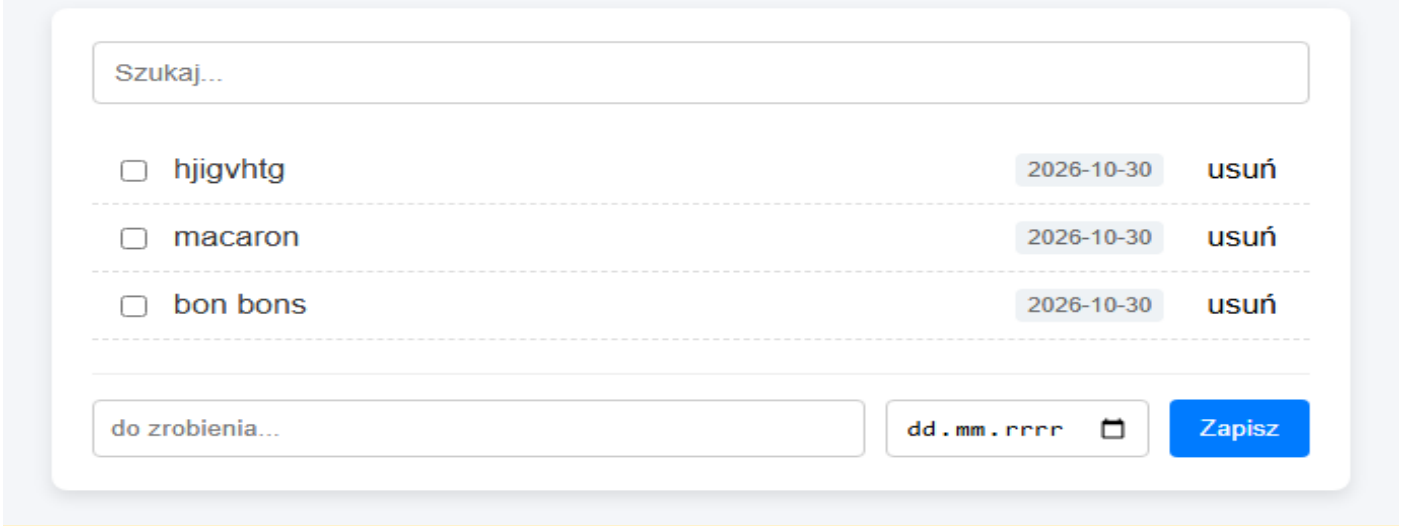
Wstaw zrzut ekranu listy po usunięciu zadania:



Punkty:	0	1

EDYCJA POZYCJI LISTY

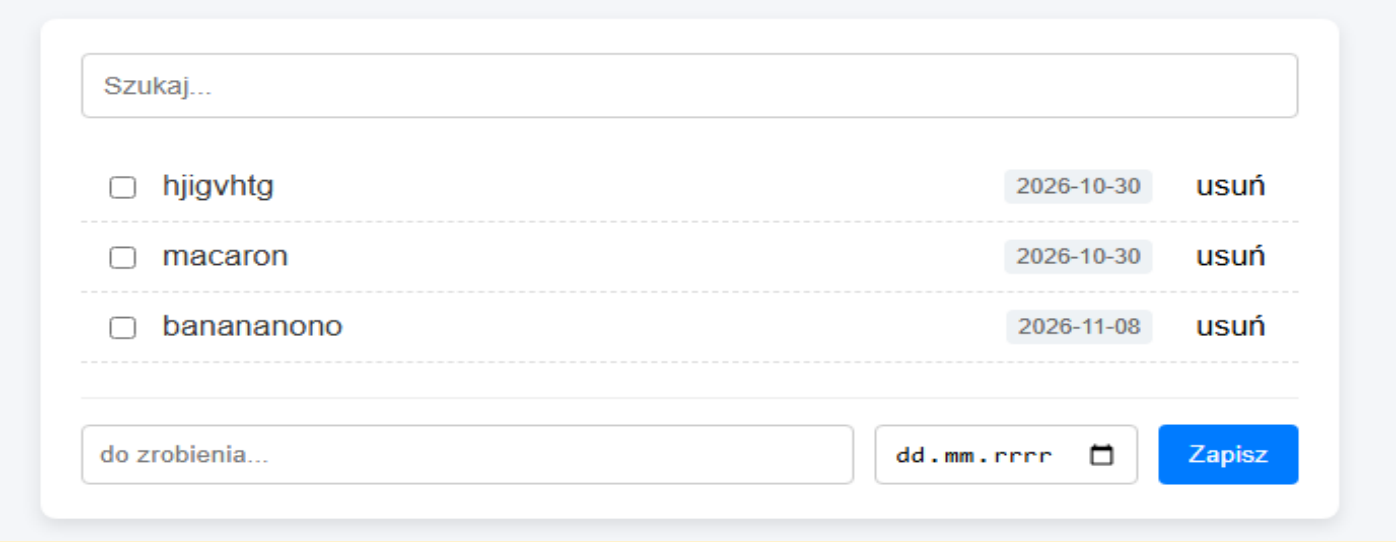
Wstaw zrzut ekranu listy przed edycją wybranego zadania:



Wstaw zrzut ekranu listy w trakcie edytowania zadania i daty:



Wstaw zrzut ekranu listy po edycji zadania i daty. Upewnij się, że dane się zapisały i zadanie jest zmienione:

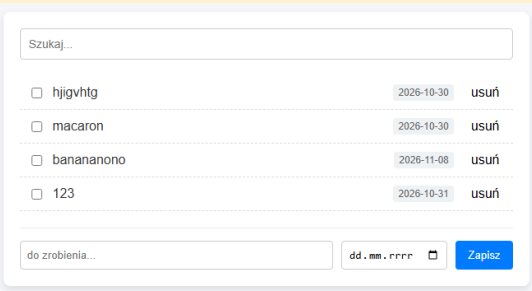


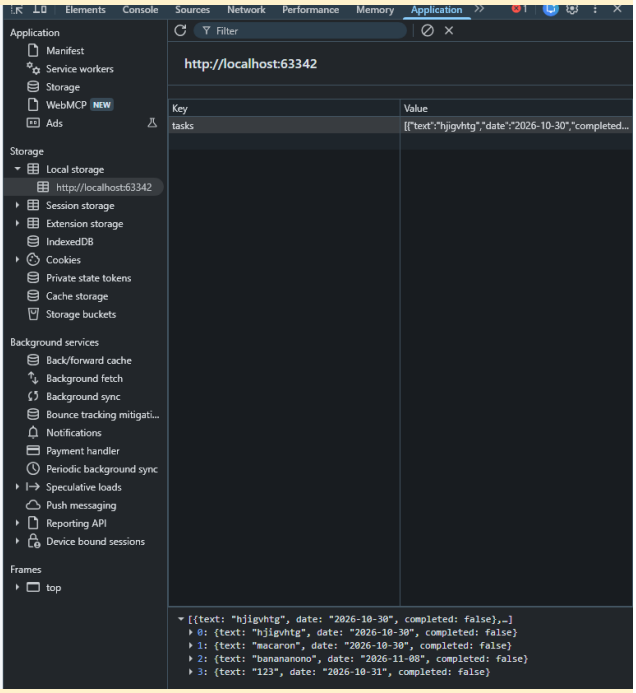
Punkty:	0	1

ODCZYT / ZAPIS LOCALSTORAGE

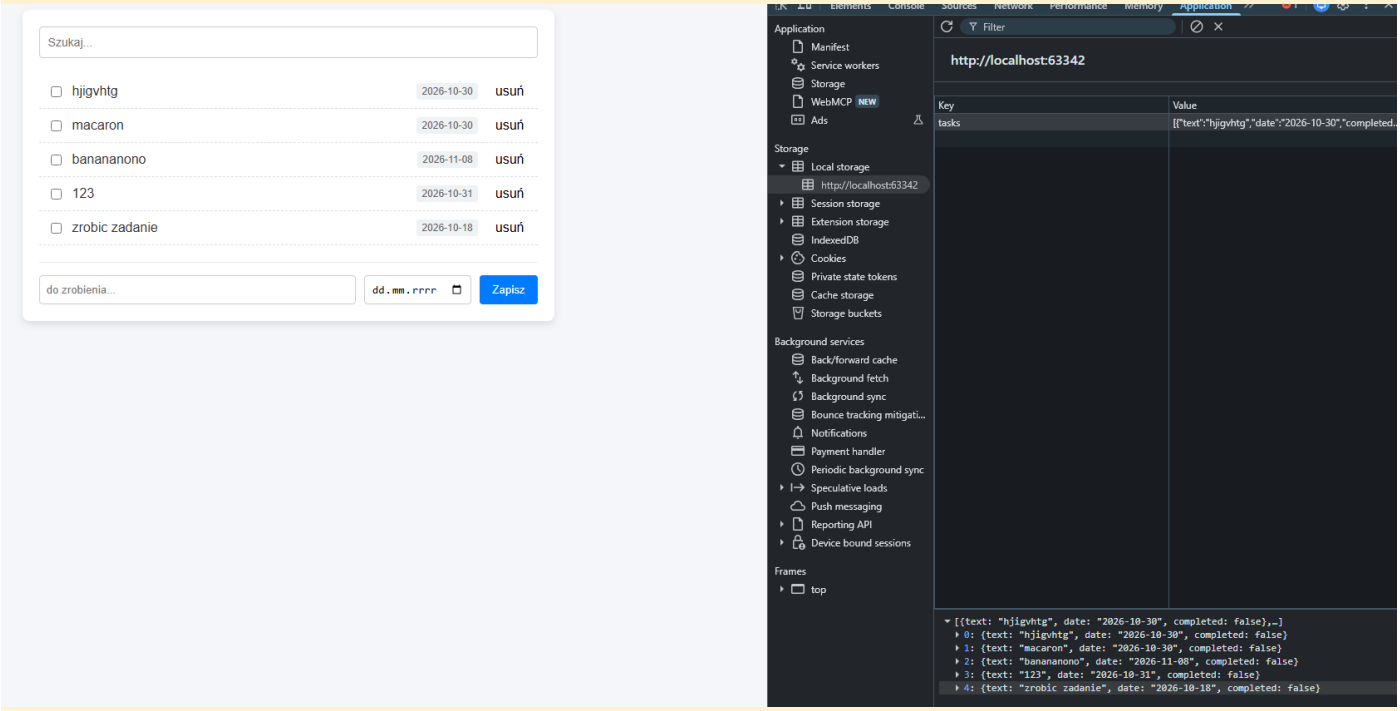
Zastosowanie klasy Todo w realizacji tego laboratorium pozwala w bardzo łatwy sposób odczytywać i zapisywać stan listy do pamięci przeglądarki. Wystarczy serializacja / deserializacja za pomocą JSON.parse() i JSON.stringify().

Wstaw zrzuty ekranu przedstawiające wygląd listy i zawartość local storage gdy na liście są pewne zadania:





Wstaw zrzuty ekranu przedstawiające wygląd listy i zawartość local storage po dodaniu nowej pozycji listy. Upewnij się, że widoczne w local storage są dane dotyczące nowego zadania:

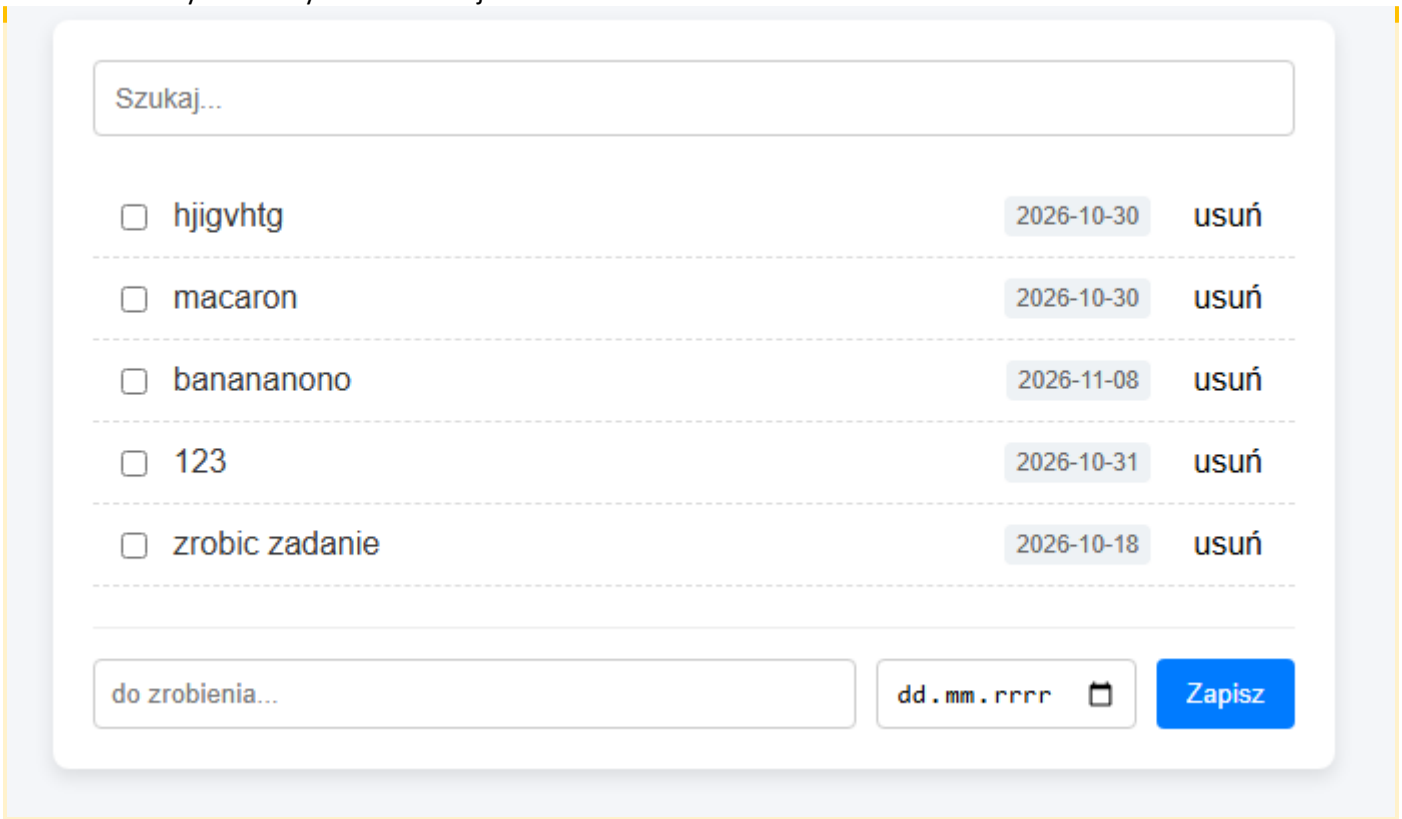


Punkty:	0	1

WYSZUKIWANIE

Na koniec zostało filtrowanie wyników. Proponowanym podejściem do tego tematu jest umieszczenie w klasie Todo właściwości `term` – frazy wyszukiwanej przez użytkownika. Następnie można utworzyć metodę `getFilteredTasks`, albo getter `filteredTasks`, która zwracać będzie te elementy tablicy `tasks`, które odpowiadają zapytaniu. Można użyć funkcji wyższego rzędu `filter()`.

Wstaw zrzut ekranu listy, gdy pole wyszukiwania jest puste:

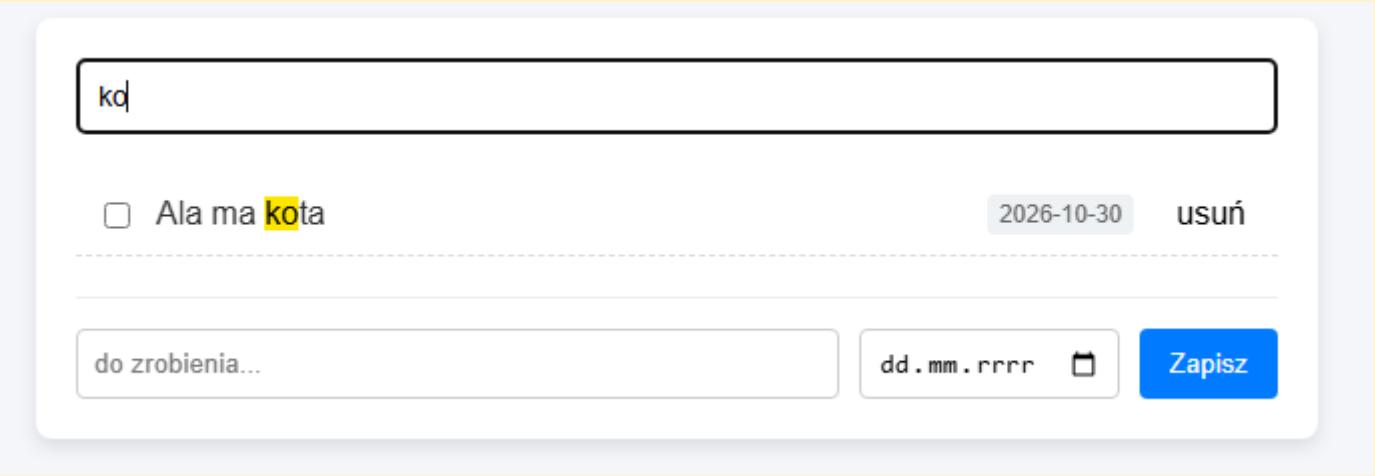


Wstaw zrzut ekranu listy, gdy w polu wyszukiwania wpisano wystarczająco dużo znaków, by zadziałało filtrowanie. Upewnij się, że chociaż 2 wyniki będą wciąż widoczne:



Punkty:	0	1

Wstaw zrzut ekranu przedstawiający podświetlenie szukanej frazy w wynikach wyszukiwania, przykładowo dla frazy `ko` i zadania `Ala ma kota` otrzymujemy: `Ala ma kota`:



Punkty:	0	1

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Upewnij się, że aktualna wersja rozwiązania jest dostępna przez GitHub Pages.

Podaj link do swojego repozytorium:

https://github.com/srm1xx/MykhailoMartynenko.github.io

https://srm1xx.github.io/MykhailoMartynenko.github.io/lab-b/boilerplate-b/

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.